

ERCOT GINR Payment Failure Monitor

End-to-End Architecture, Business Problem, Solution Design & Interview Explanation

Project Name

GINR Payment Failure Monitoring and Alerting System

Technologies Used

- Python
 - Pandas
 - Oracle Database
 - Flask (Future API Layer)
 - SMTP Email Automation
 - Scheduler (Daily Batch Processing)
 - Ollama LLM (Optional AI Summary)
 - Logging & Audit Framework
 - Jenkins
 - Git
-

1. Executive Summary

ERCOT receives Generation Interconnection Requests (GINRs) from developers requesting connection of solar, wind, battery, and thermal generation facilities to the Texas power grid.

Before engineering studies can begin, developers must pay several required fees including:

- Full Interconnection Study (FIS) Fee
- Scoping Study (SS) Fee
- Analysis Fees
- Change Request Fees

A significant number of projects experience delays due to payment-related issues.

The objective of this project was to build an automated monitoring system that continuously identifies payment failures, generates alerts, and notifies stakeholders before project schedules are impacted.

The system performs automated analysis, identifies payment anomalies, generates management reports, and sends email notifications to operations teams.

2. Business Problem

Existing Process

Payment verification was largely manual.

Operations teams had to:

- Review payment records
- Compare payment status across systems
- Investigate missing payments
- Track stale requests
- Generate manual reports

This process was time-consuming and error-prone.

Challenges

1. Unpaid Study Fees

Developers submit requests but do not complete required payments.

2. Payment Synchronization Issues

Payment gateway shows successful payment but RARF system is not updated.

3. Duplicate Payments

Developers accidentally pay multiple times.

4. Amount Mismatches

Partial payments are received.

5. Stale Requests

Requests remain pending for extended periods.

6. Lack of Visibility

Management lacks real-time payment health information.

Business Impact

- Delayed engineering studies
 - Project approval delays
 - Revenue leakage
 - Customer dissatisfaction
 - Increased manual effort
 - Compliance risks
-

3. Business Objective

The business requested a solution capable of:

Automatically Detecting

- Missing payments
- Incorrect payments
- Duplicate payments
- Stale requests

Providing

- Daily monitoring
- Automated alerts
- Executive reports
- Root cause visibility

Benefits

- Faster issue resolution
 - Reduced manual effort
 - Improved customer experience
 - Better operational visibility
-

4. Solution Overview

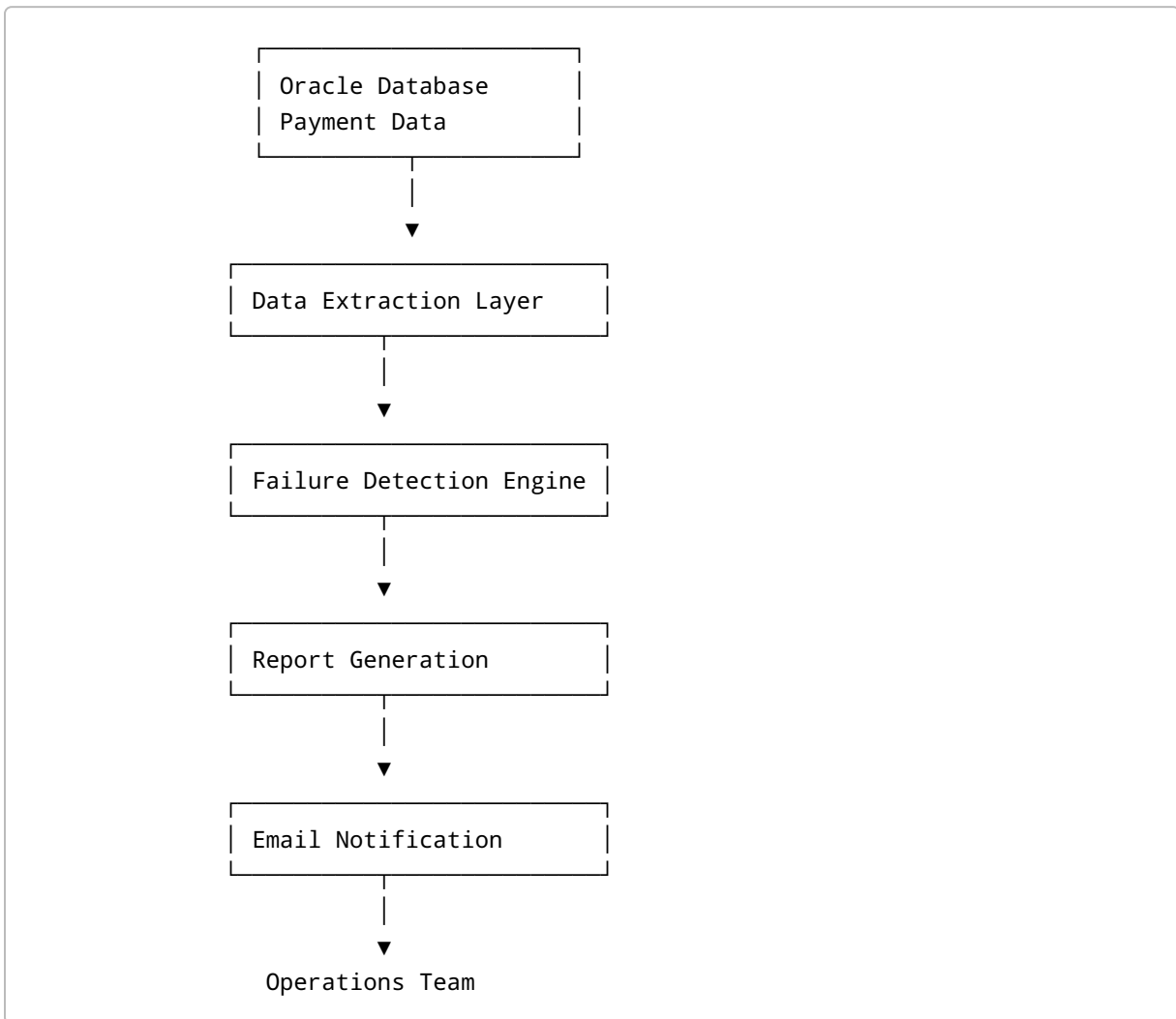
We designed an automated Payment Failure Monitoring System that analyzes payment transactions daily and identifies operational issues before they impact project timelines.

The solution consists of:

1. Data Extraction Layer
2. Payment Validation Engine

- 3. Failure Detection Engine
 - 4. Reporting Layer
 - 5. Email Notification Service
 - 6. Scheduler
 - 7. Audit & Monitoring Layer
 - 8. AI Summary Generator (Optional)
-

5. High-Level Architecture



6. Payment Failure Categories

The system detects seven business-critical failure types.

1. FIS Fee Unpaid

Problem:

Required Full Interconnection Study fee has not been paid.

Impact:

Engineering study cannot begin.

Severity:

CRITICAL

Action:

Contact developer immediately.

2. Scoping Study Fee Unpaid

Problem:

Scoping Study fee not received.

Impact:

Project cannot move forward.

Severity:

CRITICAL

Action:

Notify project owner.

3. Analysis Fee Outstanding

Problem:

Change Request analysis fee remains unpaid.

Impact:

Request remains blocked.

Severity:

HIGH

Action:

Send payment reminder.

4. Payment Synchronization Error

Problem:

Payment exists in payment gateway but application status not updated.

Impact:

False payment failure.

Severity:

HIGH

Action:

Investigate integration issue.

5. Amount Mismatch

Problem:

Developer paid less than required amount.

Impact:

Outstanding balance exists.

Severity:

HIGH

Action:

Request remaining balance.

6. Duplicate Payment

Problem:

Multiple payments detected for same request.

Impact:

Refund risk.

Severity:

MEDIUM

Action:

Review duplicate transactions.

7. Stale Pending Request

Problem:

Request remains pending for more than configured threshold.

Impact:

Project stagnation.

Severity:

MEDIUM

Action:

Follow up with customer.

7. Step-by-Step Execution Flow

Step 1 – Daily Scheduler

A scheduler runs every morning.

Example:

7:00 AM CST

Purpose:

Automated daily monitoring.

Step 2 – Data Extraction

System retrieves:

- Payment transactions
- Request status
- Fee information
- Change request data

Source:

Oracle Database

Production Query:

GINR_PAYMENT_DATA_VW

Output:

Daily Payment Dataset

Step 3 – Payment Validation

System validates:

- Payment received
- Fee amount correctness
- Payment status

- Request status

Output:

Validated Payment Records

Step 4 – Failure Detection Engine

Business rules execute against every request.

Example:

IF

FIS_FEE > 0

AND

FIS_FEE_PAID = FALSE

THEN

FIS_FEE_UNPAID

All seven rules are executed.

Output:

Failure Records

Step 5 – Severity Classification

Failures categorized as:

CRITICAL

HIGH

MEDIUM

LOW

Purpose:

Prioritize operational response.

Step 6 – Report Generation

System generates:

- Daily summary
- Failure counts
- Severity distribution
- Impact analysis

Example:

Critical Issues = 12

High Issues = 18

Medium Issues = 24

Total Failures = 54

Step 7 – AI Summary Generation (Optional)

Using Ollama LLM.

Prompt:

Summarize today's payment failures for management.

Output:

Natural language executive summary.

Example:

"12 critical payment failures were detected today. Most issues are related to unpaid FIS fees and payment synchronization problems."

Step 8 – Email Notification

Automated emails sent to:

- Operations Team
- Managers
- Support Teams

Email Includes:

- Failure Summary
 - Severity Counts
 - Recommended Actions
 - AI Generated Summary
-

Step 9 – Audit Logging

Every execution recorded.

Stored Information:

- Execution Time
- Failure Counts
- Processing Duration
- Email Status

Purpose:

Compliance and troubleshooting.

8. Deployment Architecture

Production Environment

Oracle Database ↓ Python Monitoring Engine ↓ Failure Detection Rules ↓ Report Generator ↓ Email Service ↓ Operations Team

9. Business Value Delivered

Before Implementation

- Manual payment verification
- Delayed issue detection
- Reactive support model
- No automated alerts

After Implementation

- Automated monitoring
- Early issue detection
- Faster resolution
- Reduced manual effort
- Improved visibility

Expected Benefits

- 70% reduction in manual monitoring effort
 - Faster payment issue resolution
 - Improved study processing timelines
 - Reduced operational risk
-

10. My Role in the Project

Responsibilities:

- Gathered business requirements
 - Designed monitoring architecture
 - Developed payment validation engine
 - Implemented business rule framework
 - Built reporting module
 - Developed email notification service
 - Integrated AI-based executive summaries
 - Implemented audit logging
 - Performed testing and deployment support
-

11. Future Enhancements

Machine Learning Prediction

Predict future payment failures before they occur.

LangGraph Multi-Agent System

Agent 1 – Payment Analyzer

Agent 2 – Root Cause Analyzer

Agent 3 – Email Generator

Agent 4 – Executive Reporting Agent

Dashboard

Real-time monitoring using:

- Grafana
- Power BI
- Tableau

Splunk Integration

Automated operational alerts.

12. Interview Explanation (5 Minutes)

The business problem was that ERCOT operations teams were manually monitoring payment-related issues associated with Generation Interconnection Requests. Missing fees, duplicate payments, synchronization errors, and stale requests often caused project delays and required significant manual effort to identify.

To solve this problem, we developed a Payment Failure Monitoring System using Python and Oracle data sources. The system runs daily, extracts payment information, validates transactions, and executes a rule engine that identifies seven categories of payment failures including unpaid FIS fees, unpaid scoping study fees, payment synchronization errors, duplicate payments, amount mismatches, and stale requests.

The solution classifies issues by severity, generates operational reports, and automatically sends notifications to support and management teams. We also integrated an optional Ollama-based AI summary capability that converts technical findings into executive-level summaries.

This solution significantly reduced manual monitoring effort, improved operational visibility, accelerated issue resolution, and helped prevent delays in engineering study execution.